Calm Haven Uploader — complete upload script (YouTube Data API v3, videos.insert + thumbnails.set)

Source: https://github.com/zikmanouch-oss/yt-relax

src/upload.py

```
"""Publication sur YouTube via l'API Data v3 (gratuite)."""
from __future__ import annotations

import os
import time

from google.oauth2.credentials import Credentials
from googleapiclient.discovery import build
from googleapiclient.errors import HttpError
from googleapiclient.http import MediaFileUpload

SCOPES = ["https://www.googleapis.com/auth/youtube.upload",
          "https://www.googleapis.com/auth/youtube"]


def credentials_available() -> bool:
    return all(os.environ.get(k) for k in ("YT_CLIENT_ID", "YT_CLIENT_SECRET", "YT_REFRESH_TOKEN"))


def _client():
    creds = Credentials(
        token=None,
        refresh_token=os.environ["YT_REFRESH_TOKEN"],
        client_id=os.environ["YT_CLIENT_ID"],
        client_secret=os.environ["YT_CLIENT_SECRET"],
        token_uri="https://oauth2.googleapis.com/token",
        scopes=SCOPES,
    )
    return build("youtube", "v3", credentials=creds, cache_discovery=False)


def _fit_tags(tags: list[str], limit: int = 450) -> list[str]:
    """YouTube limite les tags à 500 caractères (guillemets comptés pour les tags à espaces)."""
    out, total = [], 0
    for t in dict.fromkeys(t.strip() for t in tags if t.strip()):
        cost = len(t) + (2 if " " in t else 0) + 1
        if total + cost > limit:
            break
        out.append(t)
        total += cost
    return out


def upload(video: str, thumb: str, meta: dict, cfg: dict) -> str:
    yt = _client()
    body = {
        "snippet": {
            "title": meta["title"],
            "description": meta["description"],
            "tags": _fit_tags(meta["tags"]),
            "categoryId": str(cfg.get("category_id", 22)),
            "defaultLanguage": "en",
        },
        "status": {
            "privacyStatus": os.environ.get("PRIVACY_OVERRIDE") or cfg.get("privacy", "public"),
            "selfDeclaredMadeForKids": False,
        },
    }
    media = MediaFileUpload(video, mimetype="video/mp4", chunksize=64 * 1024 * 1024, resumable=True)
    req = yt.videos().insert(part="snippet,status", body=body, media_body=media)

    response, retries = None, 0
    while response is None:
        try:
            status, response = req.next_chunk()
            if status:
                print(f"Upload {int(status.progress() * 100)} %", flush=True)
        except HttpError as e:
            if e.resp.status in (500, 502, 503, 504) and retries < 8:
                retries += 1
                time.sleep(2 ** retries)
                continue
            raise
    video_id = response["id"]
    print(f"Vidéo publiée : https://youtu.be/{video_id}")

    try:
        yt.thumbnails().set(videoId=video_id, media_body=MediaFileUpload(thumb, mimetype="image/jpeg")).execute()
    except HttpError as e:
        print(f"Miniature non appliquée (chaîne non vérifiée ?) : {e}")

    playlist = cfg.get("playlist_id")
    if playlist:
        try:
            yt.playlistItems().insert(part="snippet", body={"snippet": {
                "playlistId": playlist,
                "resourceId": {"kind": "youtube#video", "videoId": video_id}}}).execute()
        except HttpError as e:
            print(f"Ajout à la playlist impossible : {e}")
    return video_id
```

src/main.py

```
"""Génère une vidéo de pluie réaliste et la publie sur YouTube.

Usage :
    python -m src.main                          # vidéo complète + publication
    python -m src.main --hours 0.05 --dry-run   # test de 3 min, sans publication
"""
from __future__ import annotations

import argparse
import json
import random
import shutil
import time
from datetime import datetime, timezone
from pathlib import Path

import yaml

from . import footage, metadata, soundscape, sources, visual

ROOT = Path(__file__).resolve().parent.parent
HISTORY = ROOT / "history.json"


def load_history() -> list:
    if HISTORY.exists():
        return json.loads(HISTORY.read_text(encoding="utf-8"))
    return []


def step(msg: str, t0: list) -> None:
    now = time.time()
    print(f"[{now - t0[0]:6.0f} s] {msg}", flush=True)


def main() -> None:
    ap = argparse.ArgumentParser()
    ap.add_argument("--hours", type=float, default=None)
    ap.add_argument("--dry-run", action="store_true", help="génère sans publier")
    ap.add_argument("--out", default="out")
    args = ap.parse_args()

    cfg = yaml.safe_load((ROOT / "config.yaml").read_text(encoding="utf-8"))
    hours = args.hours if args.hours is not None else float(cfg["hours"])
    seconds = int(hours * 3600)
    history = load_history()
    variant = cfg["variants"][len(history) % len(cfg["variants"])]
    theme, thunder = variant["theme"], bool(variant["thunder"])
    seed = int(time.time()) % 1_000_000
    rng = random.Random(seed)
    t0 = [time.time()]

    recent = history[-6:]
    avoid_clips = {i for h in recent for i in h.get("clip_ids", [])}
    avoid_sounds = {i for h in recent for i in h.get("sound_ids", [])}
    blocked = {int(i) for i in cfg.get("blocked_clips") or []}

    out = Path(args.out)
    assets = out / "assets"
    assets.mkdir(parents=True, exist_ok=True)
    step(f"Ambiance « {theme} », tonnerre={thunder}, {hours:g} h, graine {seed}", t0)

    # 1. Médias réels
    tcfg = cfg["themes"][theme]
    n_clips = int(cfg.get("clips_per_video", 8))
    clips = sources.video_clips(rng.choice(tcfg["video"]), n_clips, assets, rng, avoid_clips, blocked)
    for q in tcfg["video"]:
        if len(clips) >= 4:
            break
        clips += sources.video_clips(q, n_clips - len(clips), assets, rng,
                                     avoid_clips | {c["id"] for c in clips}, blocked)
    if len(clips) < 3:
        raise SystemExit("Pas assez de clips vidéo exploitables.")
    sounds = sources.freesound_sounds(rng.choice(tcfg["sound"]), int(cfg.get("sounds_per_video", 5)),
                                      assets, rng, avoid=avoid_sounds)
    if len(sounds) < 3:
        sounds += sources.freesound_sounds("rain", 5 - len(sounds), assets, rng,
                                           avoid=avoid_sounds | {s["id"] for s in sounds})
    thunders = []
    if thunder:
        thunders = sources.freesound_sounds(rng.choice(cfg["themes"]["thunder_sound"]), 3, assets, rng, min_dur=8)
    step("Médias téléchargés", t0)

    # 2. Vidéo : clips normalisés → segment bouclable
    norm = [footage.normalize(c["path"], str(assets / f"n{i}.mp4")) for i, c in enumerate(clips)]
    seg_target = min(int(cfg.get("segment_minutes", 15) * 60), max(60, seconds))
    segment = str(out / "segment.mp4")
    loop_len = footage.build_segment(norm, seg_target, segment, rng, cfg.get("video_maxrate", "3M"))
    step(f"Segment vidéo de {loop_len / 60:.1f} min prêt", t0)

    # 3. Son : mixage des enregistrements réels
    audio = str(out / "audio.m4a")
    soundscape.mix([s["path"] for s in sounds], [s["path"] for s in thunders], seconds, audio, seed)
    step("Paysage sonore mixé", t0)

    # 4. Miniature + textes
    frame = footage.frame(segment, rng.uniform(5, max(6, loop_len - 5)), str(out / "frame.png"))
    meta = metadata.build(theme, thunder, hours, seed, clips, sounds + thunders)
    meta["clips"] = [{"id": c["id"], "page": c.get("page", "")} for c in clips]
    thumb = str(out / "thumb.jpg")
    visual.thumbnail(frame, meta["thumb_line1"], meta["thumb_line2"], thumb)
    (out / "metadata.json").write_text(json.dumps(meta, ensure_ascii=False, indent=2), encoding="utf-8")
```

```
    shutil.rmtree(assets, ignore_errors=True)  # libère le disque avant l'assemblage final

    # 5. Assemblage final (copie directe, sans réencodage)
    video = str(out / "video.mp4")
    footage.final_mux(segment, audio, seconds, video)
    step(f"Vidéo finale : {Path(video).stat().st_size / 1e9:.2f} Go", t0)

    if args.dry_run:
        print("Mode test : pas de publication.")
        return

    from . import upload
    if not upload.credentials_available():
        raise SystemExit("Identifiants YouTube absents (secrets YT_CLIENT_ID, YT_CLIENT_SECRET, YT_REFRESH_TOKEN).")
    Path(segment).unlink(missing_ok=True)
    video_id = upload.upload(video, thumb, meta, cfg)
    history.append({
        "date": datetime.now(timezone.utc).isoformat(timespec="seconds"),
        "video_id": video_id,
        "title": meta["title"],
        "hours": hours,
        "seed": seed,
        "theme": theme,
        "thunder": thunder,
        "clip_ids": [c["id"] for c in clips],
        "sound_ids": [s["id"] for s in sounds],
    })
    HISTORY.write_text(json.dumps(history, ensure_ascii=False, indent=2), encoding="utf-8")
    step("Terminé", t0)


if __name__ == "__main__":
    main()
```

.github/workflows/publish.yml

```
name: Publier une vidéo pluie

on:
  schedule:
    - cron: "17 18 * * 1,3,5"   # lundi, mercredi, vendredi à 18:17 UTC
  workflow_dispatch:
    inputs:
      dry_run:
        description: "Mode test (génère sans publier)"
        type: boolean
        default: true
      hours:
        description: "Durée en heures (vide = config.yaml ; test = 0.05)"
        type: string
        default: ""
      privacy:
        description: "Forcer la confidentialité (vide = config.yaml) : private | unlisted | public"
        type: string
        default: ""
      task:
        description: "Tâche"
        type: choice
        options: [video, branding]
        default: video
      queries:
        description: "branding : recherches séparées par ;"
        type: string
        default: ""

permissions:
  contents: write

jobs:
  branding:
    if: inputs.task == 'branding'
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-python@v5
        with:
          python-version: "3.12"
      - run: pip install -q requests Pillow
      - name: Chercher des photos CC0
        run: |
          IFS=';' read -ra Q <<< "${{ inputs.queries }}"
          python scripts/branding_candidates.py "${Q[@]}"
      - name: Enregistrer
        run: |
          git config user.name "yt-relax-bot"
          git config user.email "41898282+github-actions[bot]@users.noreply.github.com"
          git add branding
          git diff --cached --quiet || git commit -m "Images candidates pour la chaîne"
          git push

  publish:
    if: inputs.task != 'branding'
    runs-on: ubuntu-latest
    timeout-minutes: 340
    concurrency:
      group: publish
      cancel-in-progress: false
    env:
      YT_CLIENT_ID: ${{ secrets.YT_CLIENT_ID }}
      YT_CLIENT_SECRET: ${{ secrets.YT_CLIENT_SECRET }}
      YT_REFRESH_TOKEN: ${{ secrets.YT_REFRESH_TOKEN }}
```

```
      PIXABAY_API_KEY: ${{ secrets.PIXABAY_API_KEY }}
      FREESOUND_API_KEY: ${{ secrets.FREESOUND_API_KEY }}
      PRIVACY_OVERRIDE: ${{ inputs.privacy }}
    steps:
      - name: Choisir le mode
        id: mode
        run: |
          if [ -z "$PIXABAY_API_KEY" ] || [ -z "$FREESOUND_API_KEY" ]; then
            echo "mode=skip" >> "$GITHUB_OUTPUT"
            echo "::warning::Secrets PIXABAY_API_KEY / FREESOUND_API_KEY manquants."
          elif [ "${{ inputs.dry_run }}" = "true" ]; then
            echo "mode=test" >> "$GITHUB_OUTPUT"
            echo "hours=${{ inputs.hours || '0.05' }}" >> "$GITHUB_OUTPUT"
          elif [ -n "$YT_CLIENT_ID" ] && [ -n "$YT_CLIENT_SECRET" ] && [ -n "$YT_REFRESH_TOKEN" ]; then
            echo "mode=publish" >> "$GITHUB_OUTPUT"
            echo "hours=${{ inputs.hours }}" >> "$GITHUB_OUTPUT"
          else
            echo "mode=skip" >> "$GITHUB_OUTPUT"
            echo "::notice::Identifiants YouTube pas encore configurés : publication ignorée."
          fi

      - uses: actions/checkout@v4
        if: steps.mode.outputs.mode != 'skip'

      - uses: actions/setup-python@v5
        if: steps.mode.outputs.mode != 'skip'
        with:
          python-version: "3.12"
          cache: pip

      - name: Installer les outils
        if: steps.mode.outputs.mode != 'skip'
        timeout-minutes: 15
        run: |
          if ! command -v ffmpeg >/dev/null || ! command -v ffprobe >/dev/null; then
            for i in 1 2 3; do
              timeout 300 sudo apt-get -o Acquire::Retries=3 -o Acquire::http::Timeout=30 update -qq &&
              timeout 300 sudo apt-get -o Acquire::Retries=3 install -y -qq --no-install-recommends ffmpeg fonts-dejavu-core &&
break
              echo "Nouvel essai d'installation ($i)…"; sleep 10
            done
          fi
          ffmpeg -version | head -1
          pip install -q -r requirements.txt

      - name: Préparer l'espace disque
        if: steps.mode.outputs.mode != 'skip'
        run: |
          sudo mkdir -p /mnt/work && sudo chown "$USER" /mnt/work
          ln -s /mnt/work out
          df -h / /mnt

      - name: Générer (et publier)
        if: steps.mode.outputs.mode != 'skip'
        run: |
          ARGS="--out out"
          [ -n "${{ steps.mode.outputs.hours }}" ] && ARGS="$ARGS --hours ${{ steps.mode.outputs.hours }}"
          [ "${{ steps.mode.outputs.mode }}" = "test" ] && ARGS="$ARGS --dry-run"
          python -m src.main $ARGS

      - name: Extrait léger (2 min autour du raccord de boucle)
        if: steps.mode.outputs.mode == 'test'
        run: |
          DUR=$(ffprobe -v error -show_entries format=duration -of csv=p=0 out/video.mp4)
          SEG=$(ffprobe -v error -show_entries format=duration -of csv=p=0 out/segment.mp4)
          START=$(python3 -c "d=$DUR; s=$SEG; print(max(0, min(s - 60, d - 120)))")
          ffmpeg -v error -y -ss "$START" -i out/video.mp4 -t 120 -c:v libx264 -preset veryfast -crf 28 -vf scale=1280:-2 -c:a
aac -b:a 128k out/extrait.mp4
          ls -lh out/extrait.mp4

      - name: Aperçu du test
        if: steps.mode.outputs.mode == 'test'
        uses: actions/upload-artifact@v4
        with:
          name: apercu
          path: |
            out/extrait.mp4
            out/thumb.jpg
            out/frame.png
            out/metadata.json
          retention-days: 3

      - name: Enregistrer l'historique
        if: steps.mode.outputs.mode == 'publish'
        run: |
          git config user.name "yt-relax-bot"
          git config user.email "41898282+github-actions[bot]@users.noreply.github.com"
          git add history.json
          git diff --cached --quiet || git commit -m "Historique : nouvelle vidéo publiée"
          git push
```